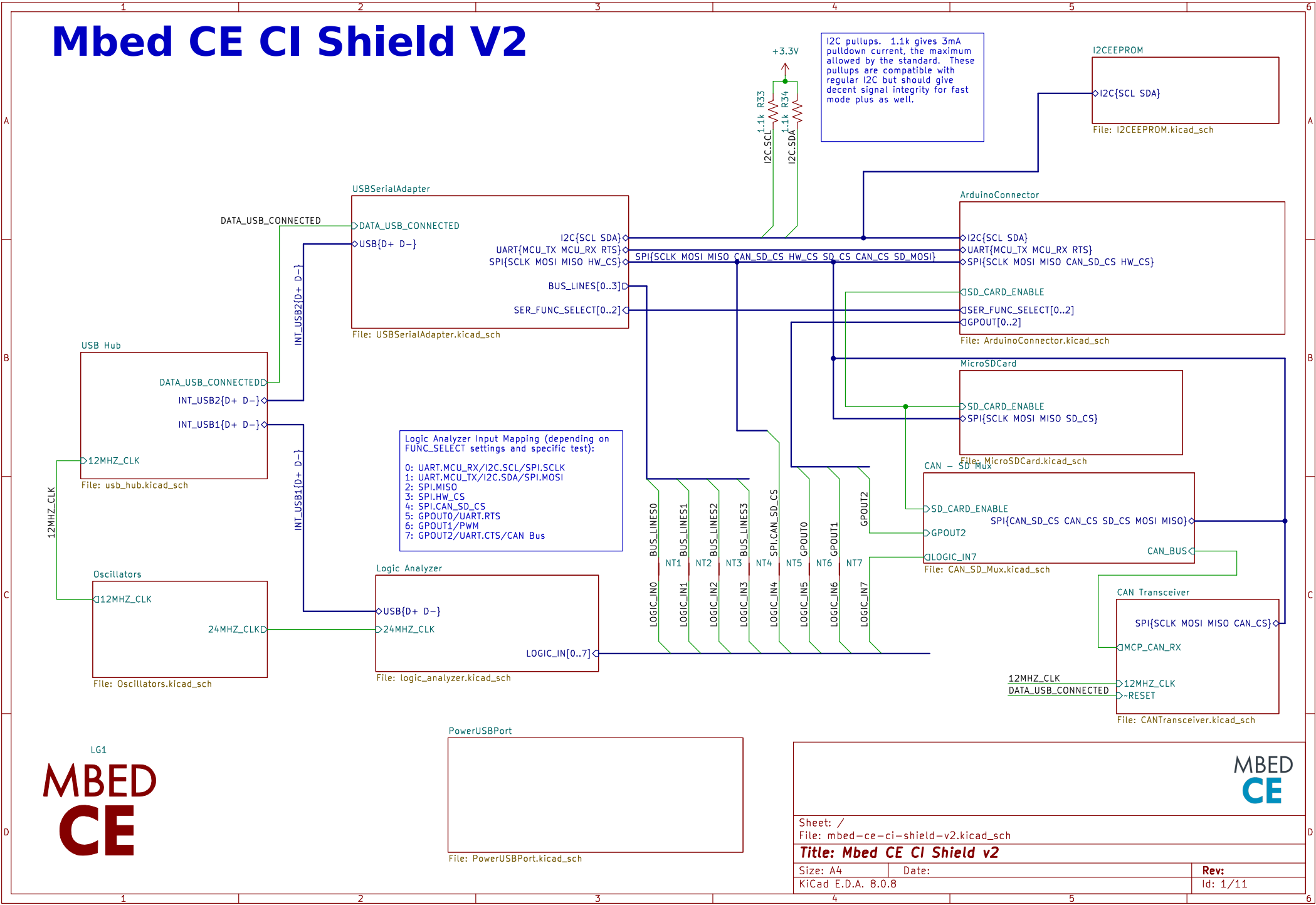
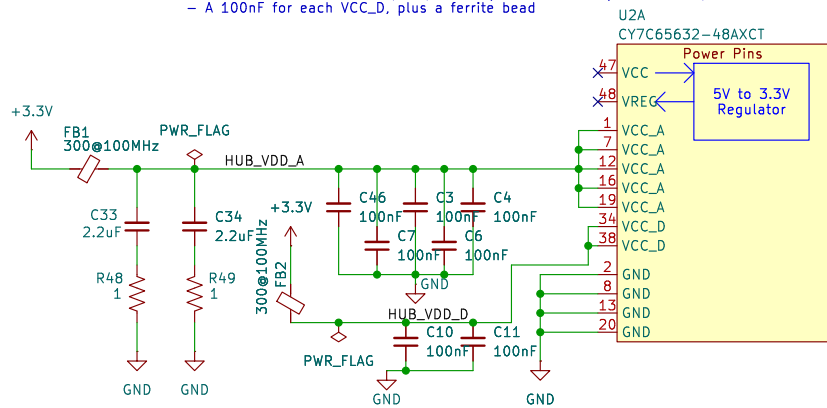


Mbed CE CI Shield V2

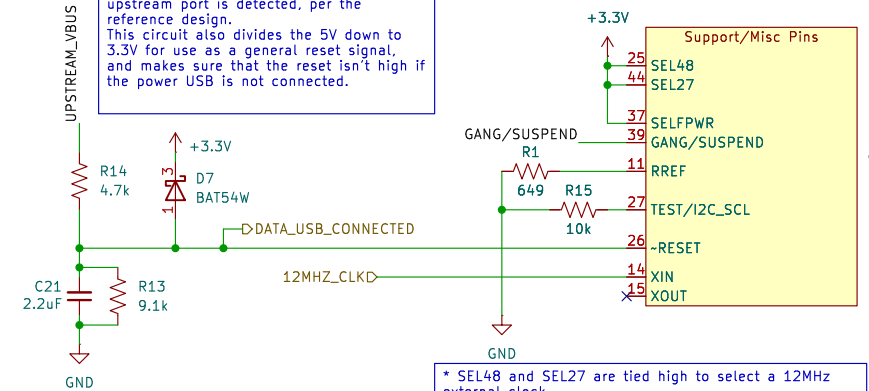


Based on the reference design and review feedback, I assume that needed debouncing is:

- 1x 10uF and 1x 1uF on VREG (which I assume is needed only if the internal regulator is used)
- A 100nF for each VCC_A pin, plus 2x (2.2uF + 1 Ohm) on VCC_A, plus a ferrite bead
- A 100nF for each VCC_D, plus a ferrite bead

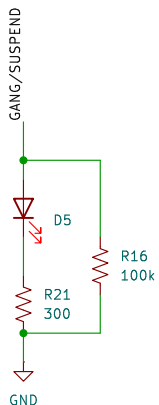


RC timer is used to bring the hub chip's reset line high ~10ms after VBUS from the upstream port is detected, per the reference design. This circuit also divides the 5V down to 3.3V for use as a general reset signal, and makes sure that the reset isn't high if the power USB is not connected.

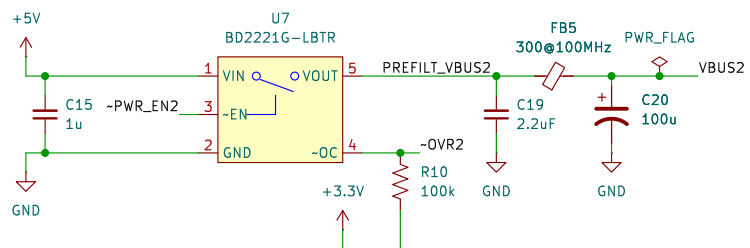
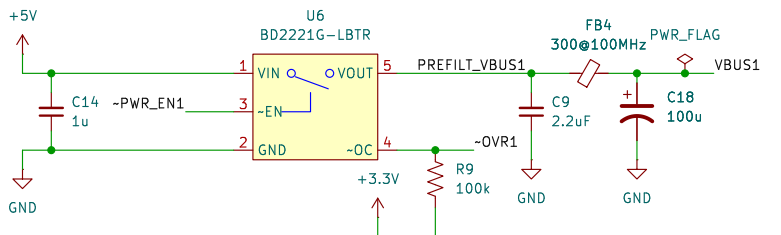


- * SEL48 and SEL27 are tied high to select a 12MHz external clock.
- * SELF_PWR is tied high to indicate that the hub is not powered by the upstream USB port
- * TEST/I2C_SCL is pulled down to disable test mode

Hub Active LED: Lights up when hub is active (NOT in suspend mode). 100k pulldown is a strapping setting to set that the PWR pins can act independently.



External port power switches



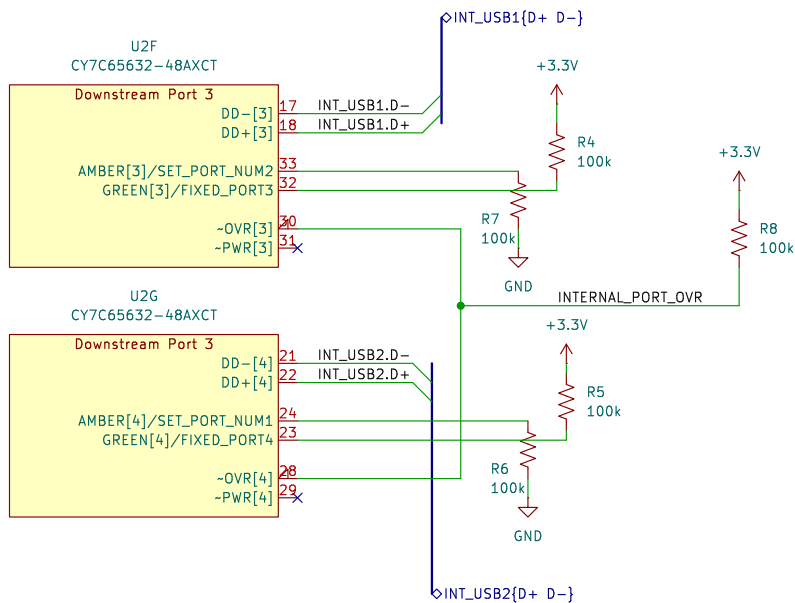
Internal Ports

Internal ports. These ports are used to provide USB to devices inside the CI shield. Due to this, they lack ESD protection or their own power supply.

The FIXED_PORT pins are strapped to high to indicate to the hub that these devices are non-removable.

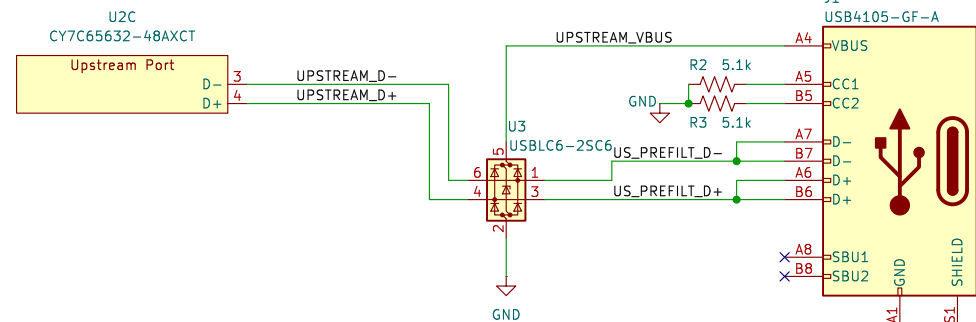
The SET_PORT_NUM[2:1] pins are strapped to 00 to indicate that all 4 ports are in use.

The OVR pins are tied to high to disable overcurrent detection.

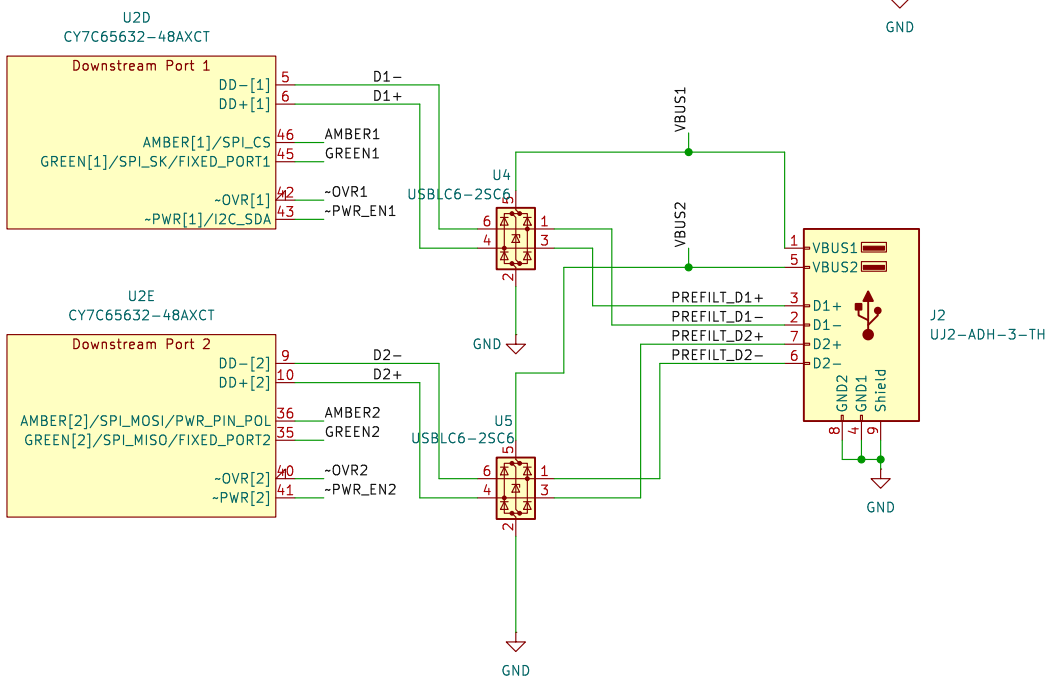


Upstream Port

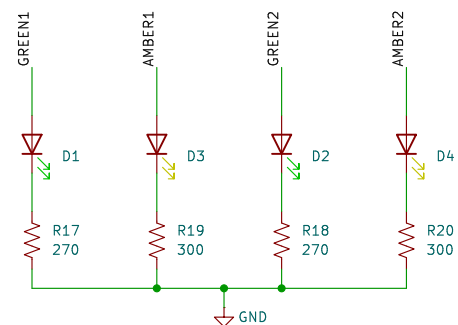
Upstream port (USB-C, connects to host computer). This uses the USB connector setup from here: <https://hackaday.com/2022/12/06/usb-c-introduction-for-hackers/> This requests 5V@1.5A for our board.



External Ports



Port Indicators



Note: For the GREEN pins, we rely on the internal pulldown resistor to strap them as low. This configures port 1 and port 2 as removable.

Also, for AMBER2, we allow this to be pulled low internally to strap the -PWR[] pins to active-low polarity.

USB hub with 2 internal and 2 external ports.

Sheet: /USB Hub/
File: usb_hub.kicad_sch

Title: USB Hub

Size: A3
KiCad E.D.A. 8.0.8

Date:

Rev:
Id: 2/11

MBED
CE

Note: the CY7C65211 datasheet does not say what to connect the exposed pad to, but the hardware design section of the EVK datasheet says to connect it to ground.
Neither says anything concrete about debouncing capacitors for the 3.3V power pins... and then the EVK schematic itself goes ham and has FIVE debouncing capacitors for the two pins. Ima be a bit more mellow and just stick a 0.1uF on each pin.

Note: SCB_0 and SCB_1 are never needed at the same time, so to avoid needing a 3rd switch IC, we jumper them together. This requires GPIO_2 and GPIO_6 to be set as tristated for when these pins are not used as SCB pins

Note: WAKEUP pin is active high by default (per EVK datasheet section 3.3.2), so we ground it to disable the wakeup functionality.

The four post-multiplexing bus lines are tapped for the logic analyzer

Analog multiplexer allows the CY7C65211 to be connected to different pins on the MCU depending on what the test suite wants to test.
FUNC_SELECT | Function

000 | UART
001 | I²C
010 | SPI
1xx | Hi-Z

Ensure that the default setting is 100 (Hi-Z)

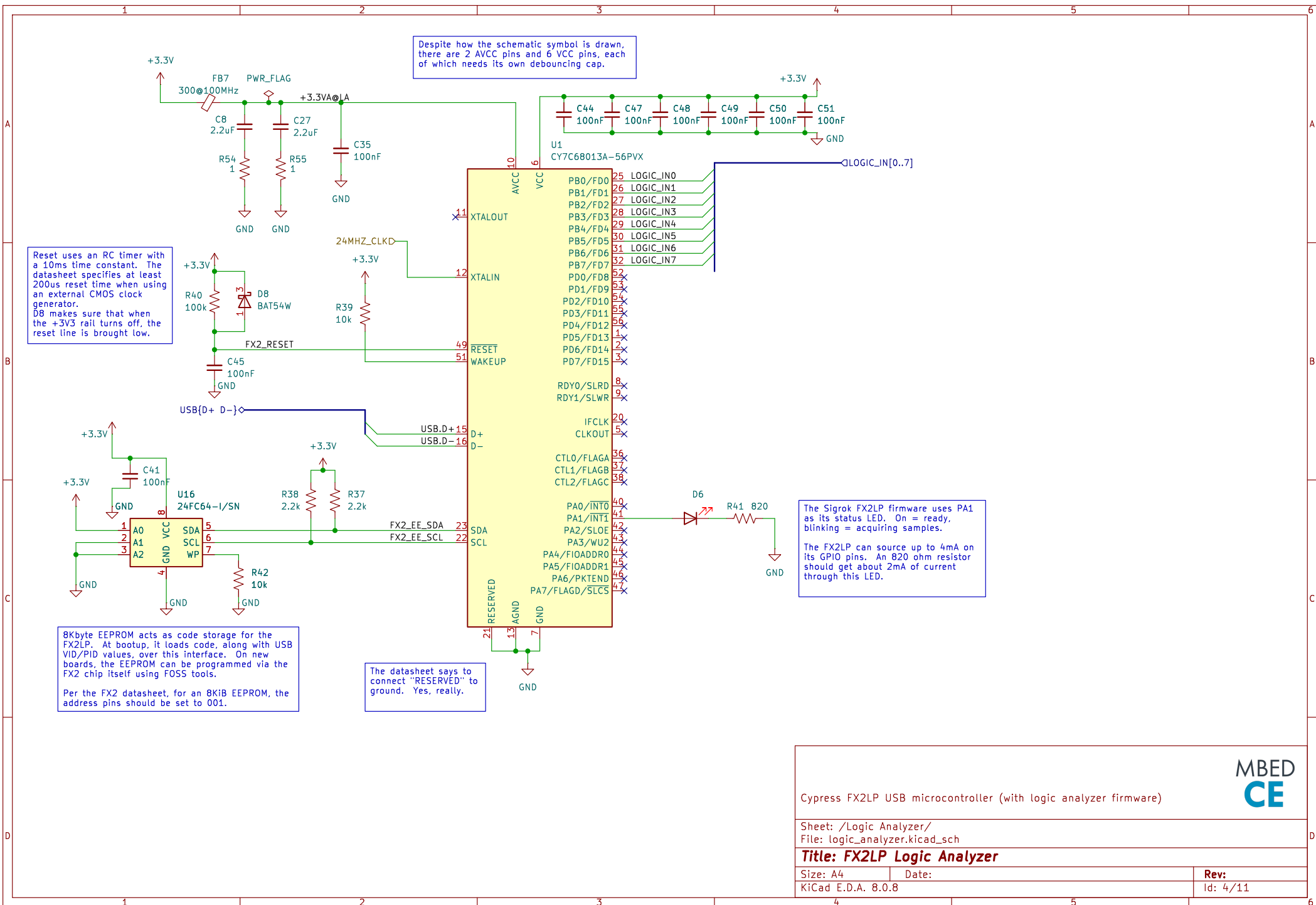
Table 14. Serial Communication Block Configuration

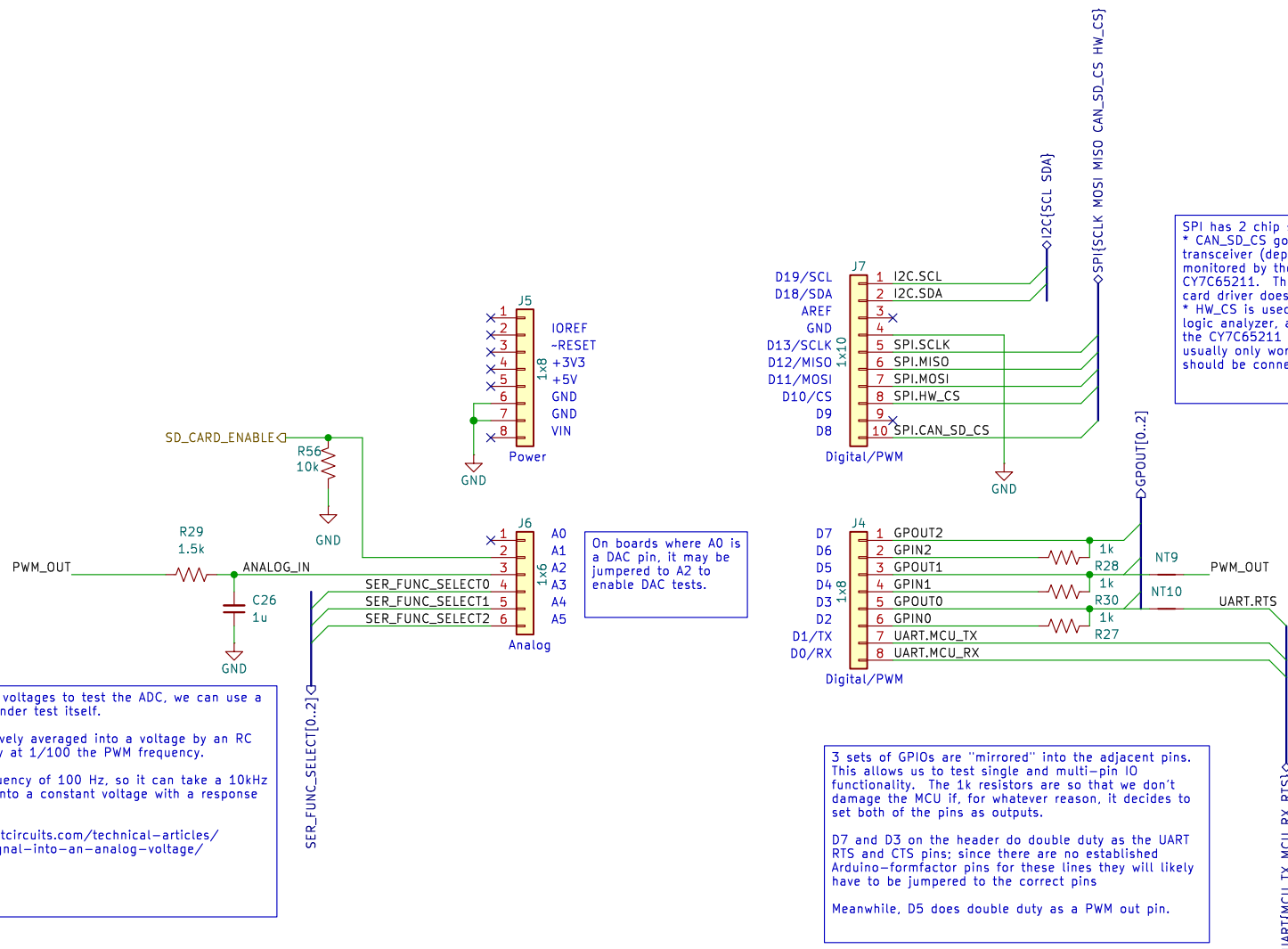
Pin	Serial Port	Mode 0*	Mode 1	Mode 2	Mode 3	Mode 4	Mode 5	Mode 6
		6-pin UART	4-pin UART	2-pin UART	SPI Master	SPI Slave	I2C Master	I2C Slave
1	SCB_0	RxD	RxD	RxD	GPIO_6	GPIO_6	GPIO_6	GPIO_6
20	SCB_1	DSR#	GPIO_2	GPIO_2	SSEL_OUT	SSEL_IN	GPIO_2	GPIO_2
21	SCB_2	RTS#	RTS#	GPIO_3	MISO_IN	MISO_OUT	SCL_OUT	SCL_IN
22	SCB_3	CTS#	CTS#	GPIO_4	MOSI_OUT	MOSI_IN	SDA	SDA
23	SCB_4	TxD	TxD	TxD	SCLK_OUT	SCLK_IN	GPIO_5	GPIO_5
2	SCB_5	DTR#	GPIO_7	GPIO_7	GPIO_7	GPIO_7	GPIO_7	GPIO_7

*Note: The device is configured in Mode 0 as the default. Other modes can be configured using the configuration utility provided by Cypress.

GPIO
SCB

Note: for serial flow control we connect UART.RTS (from the MCU) to -CTS from the CY7C65211. This allows the Mbed MCU to tell the serial adapter to stop sending by taking its RTS pin high (deasserted).
As for UART.CTS from the MCU, that is looped back to another GPIO on the ArduinoConnector sheet. We can use that GPIO to control whether the UART peripheral is seeing CTS asserted or deasserted.





Arduino Uno headers that connect to all of the board's functions

Sheet: /ArduinoConnector/
File: ArduinoConnector.kicad_sch

Title: Arduino Connector

Size: A4

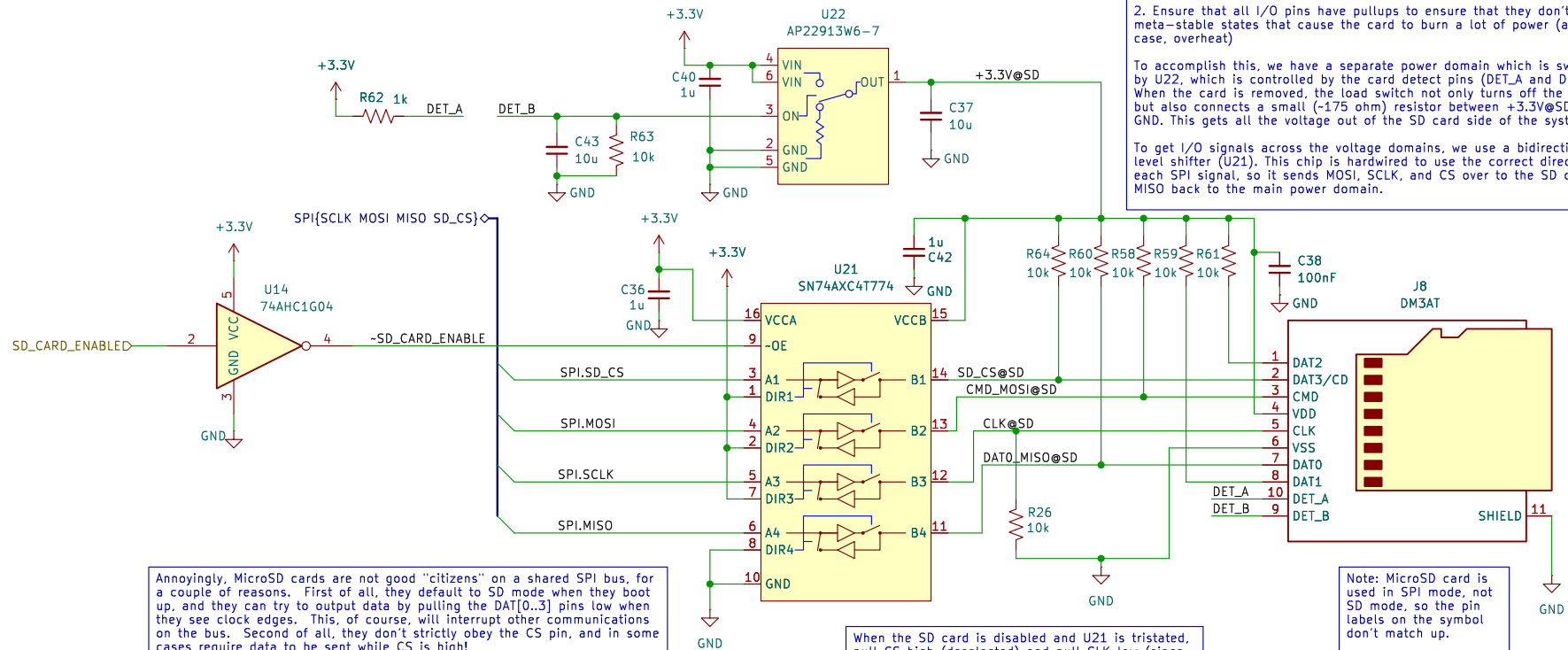
Date:

KiCad E.D.A. 8.0.8

Rev:

Id: 5/11

MBED
CE



My friend Kate H gave some advice about how to best protect MicroSD cards. Basically it boils down to:

1. Ensure that there is no power on any of the pins if the card is being removed. This can easily kill SD cards, especially if GND disconnects first so some I/O pins are at a negative voltage relative to the SD card.
2. Ensure that all I/O pins have pullups to ensure that they don't enter meta-stable states that cause the card to burn a lot of power (and, worst case, overheat)

To accomplish this, we have a separate power domain which is switched by U22, which is controlled by the card detect pins (DET_A and DET_B). When the card is removed, the load switch not only turns off the power, but also connects a small (~175 ohm) resistor between +3.3V@SD and GND. This gets all the voltage out of the SD card side of the system ASAP.

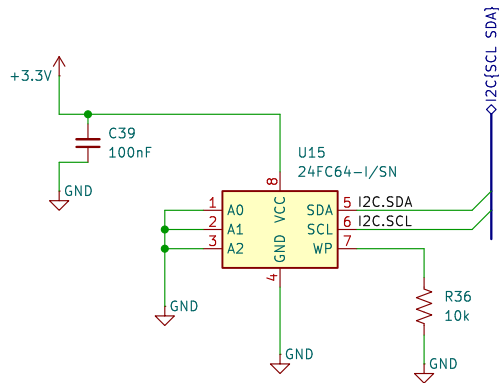
To get I/O signals across the voltage domains, we use a bidirectional level shifter (U21). This chip is hardwired to use the correct directions for each SPI signal, so it sends MOSI, SCLK, and CS over to the SD card and MISO back to the main power domain.

Annoyingly, MicroSD cards are not good "citizens" on a shared SPI bus, for a couple of reasons. First of all, they default to SD mode when they boot up, and they can try to output data by pulling the DAT[0..3] pins low when they see clock edges. This, of course, will interrupt other communications on the bus. Second of all, they don't strictly obey the CS pin, and in some cases require data to be sent while CS is high! So, to fix this, a separate line is used to connect or disconnect the card from the bus entirely by tristating the level shifter. This makes sure it's disconnected from the bus and can't cause trouble when we aren't actively using it.

When the SD card is disabled and U21 is tristated, pull CS high (deselected) and pull CLK low (since that's the idle value). This ensures that the SD card won't get any noise that looks like SPI or SD data.

Additionally, pull all of the DAT pins high because the SD card expects this and not doing it can lead to high current consumption. See <https://electronics.stackexchange.com/a/39578>

Note: MicroSD card is used in SPI mode, not SD mode, so the pin labels on the symbol don't match up.



This I2C EEPROM provides something to test an MCU's I2C implementation against. There are huge numbers of I2C EEPROMs out there, but I chose this specific one because it supports all three bus speeds in common use: 100kHz, 400kHz, and 1MHz (Fast Mode Plus). Additionally, it's cheap, seems reasonably available, and worked on the previous revision of the board.

If this specific part becomes hard to find, there should be other pin-compatible options, as this form factor of EEPROM is sort of a de-facto standard

MBED
CE

EEPROM for the board to talk to over I2C

Sheet: /I2CEEPROM/
File: I2CEEPROM.kicad_sch

Title: I2C EEPROM

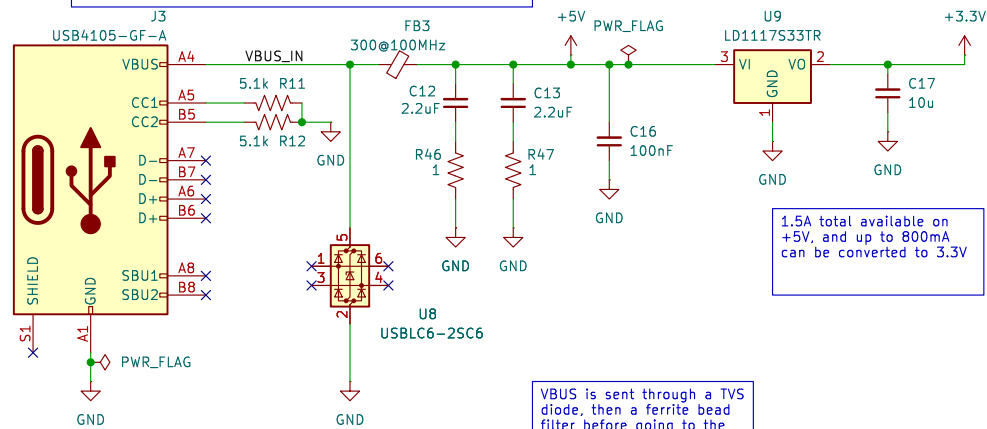
Size: A5 Date:
KiCad E.D.A. 8.0.8

Rev:
Id: 7/11

Power Input USB Port

This port supplies power to the board but not data. A separate port is used to avoid demanding too much power from the host, which might be a single-board computer without too much power to share with USB devices.

3.3V LDO



Power supply for the board using a second (power only) USB port. Intended to connect to a multi port USB power brick.



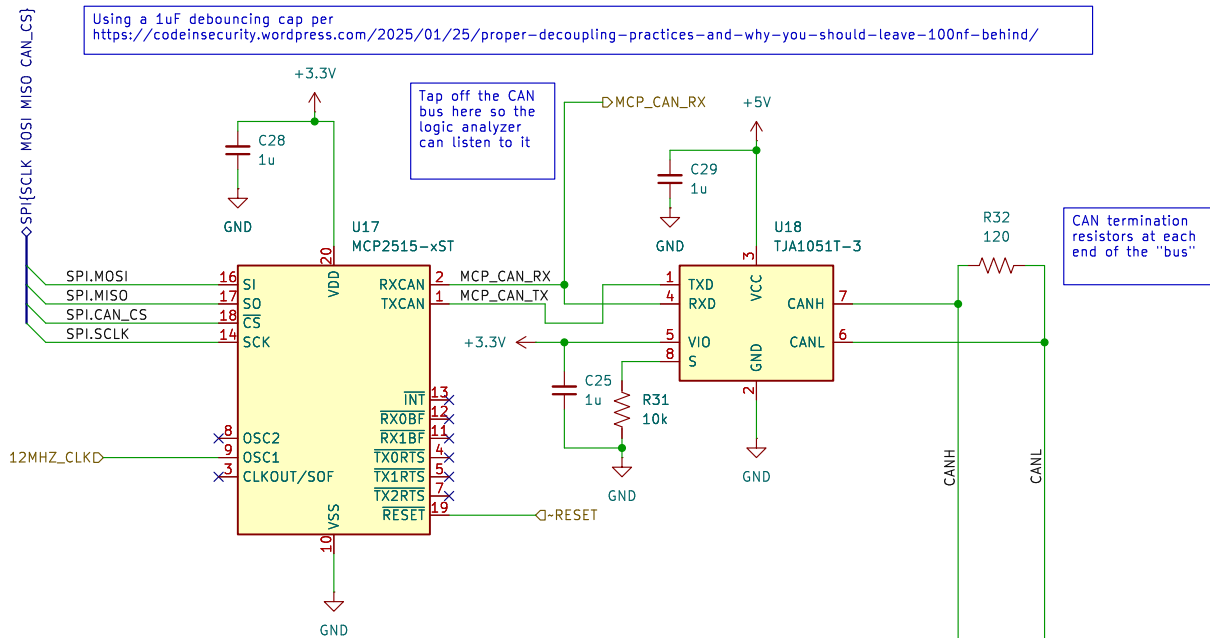
Sheet: /PowerUSBPort/
File: PowerUSBPort.kicad_sch

Title: Power Input USB Port

Size: A4
KiCad E.D.A. 8.0.8

Date:

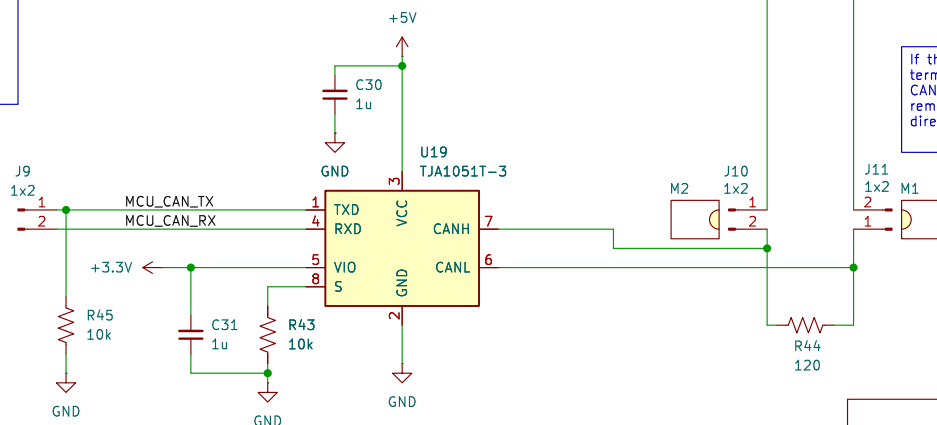
Rev:
Id: 8/11



Design adapted from the Adafruit board for the MCP2515:
<https://github.com/adafruit/Adafruit-CAN-Bus-FeatherWing-PCB/tree/main>

This chip has a register adjustable oscillator frequency, and can actually use the 12MHz clock available elsewhere. However, I haven't been able to find a driver that supports that clock frequency (drivers seem to rely on precalculated bit timings). Oh well...

There are no standard pinouts for CAN on the arduino form factor. So, we just stick some headers on, and the user must connect jumper wires to the right pins



Miniature CAN bus with an SPI-CAN adapter and a second CAN PHY for the MCU to use.



Sheet: /CAN Transceiver/
 File: CANTransceiver.kicad_sch

Title: CAN Transceiver

Size: A4

Date:

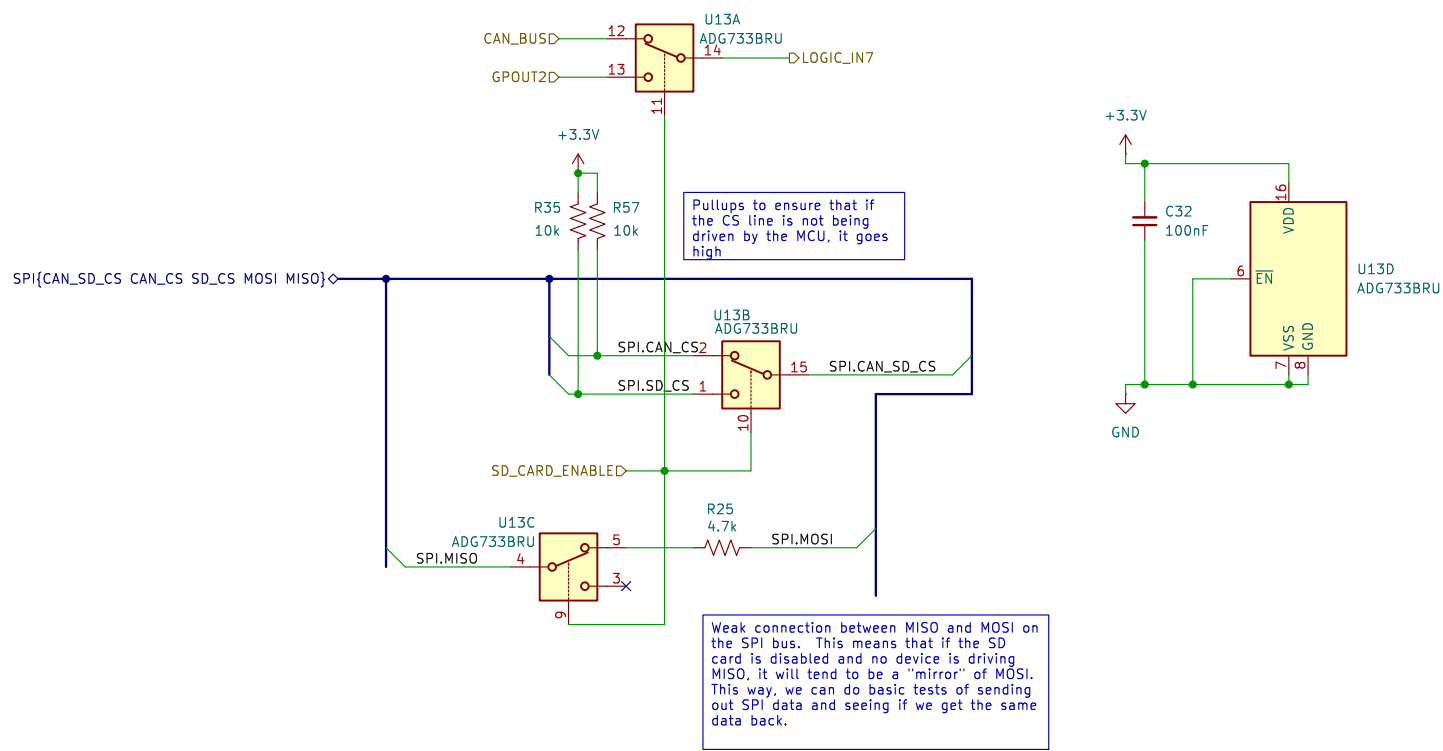
KiCad E.D.A. 8.0.8

Rev:

Id: 9/11

This sheet is responsible for:

- * Switching CAN_SD_CS to connect to either the SD card or the CAN transceiver CS line
- * Creating input 7 of the logic analyzer by switching between GPOUT2 and CAN_LISTEN
- * Switching SPI.MISO to connect to the mirror resistor



Mux for selecting CAN or SD signals.



Sheet: /CAN - SD Mux/
File: CAN_SD_Mux.kicad_sch

Title:

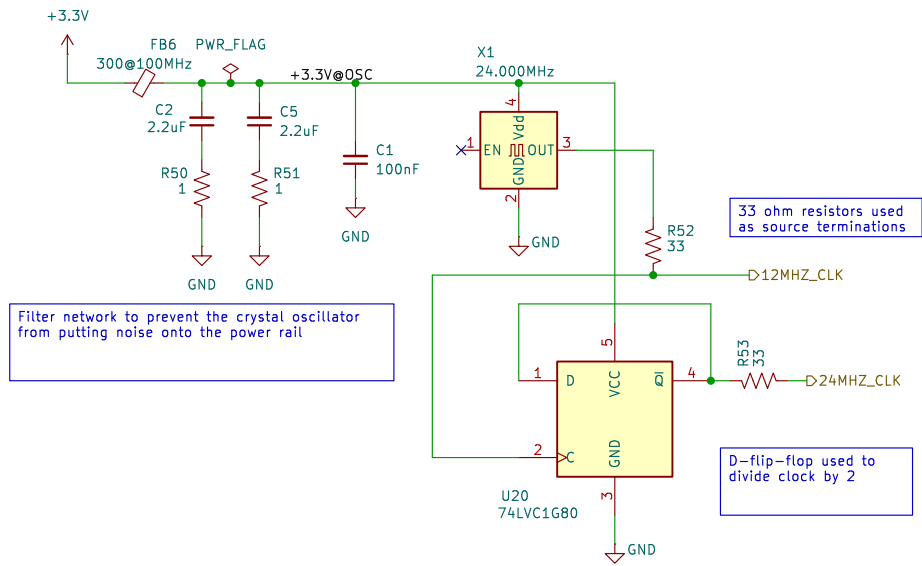
Size: A4

Date:

Rev:

KiCad E.D.A. 8.0.8

Id: 10/11



Sheet: /Oscillators/ File: Oscillators.kicad_sch		
Title:		
Size: A4	Date:	Rev:
KiCad E.D.A. 8.0.8	Id: 11/11	